

Design and Implementation of a Lightweight Multi-Protocol IoT Platform

Micke Sarro
University of Turku
Turku, Finland
mosarr@utu.fi

Abstract— The rapid expansion of the Internet of Things (IoT) requires robust, scalable middleware to bridge the gap between physical edge devices and cloud-based visualization platforms. This paper details the design and implementation of a lightweight IoT platform that facilitates core components, including device registration, local data ingestion, persistent storage, and data forwarding. The platform supports multiple application-level protocols simultaneously (HTTP, MQTT, and CoAP) utilizing a decoupled architecture to ensure non-blocking, reliable data flow to a containerized ThingsBoard dashboard.

Keywords—IoT, Middleware, Multi-Protocol, MQTT, CoAP, Flask, ThingsBoard

I. INTRODUCTION & BACKGROUND

A. Project Overview

The rapid expansion of the Internet of Things (IoT) requires robust, scalable middleware to bridge the gap between physical edge devices and cloud-based visualization platforms. The objective of this project is to design and implement a lightweight IoT platform that facilitates all the core components, including device registration, local data ingestion, information storage, and data forwarding. The platform also supports multiple application-level protocols simultaneously, specifically HTTP, MQTT, and CoAP.

The complete source code and configuration for this platform are available at: <https://github.com/mickesarro/IoT-platform-project/tree/main>

B. Project Requirements

During the initial planning phase, several core requirements were established to guide the system's development. The platform needed to perform the following operations: register new devices dynamically, receive telemetry from physical or simulated sensors (e.g., via Postman), provide capabilities for data retrieval, and support at least two distinct IoT communication protocols.

To accelerate the implementation process and ensure reliability, a stack of open-source tools was selected. Python's Flask framework was chosen as the primary programming language for the middleware. For data handling, SQLite was chosen due to its ease of integration with the Flask web framework [1]. For the IoT protocols, Eclipse Mosquitto handles MQTT traffic, while the aiocoap library manages CoAP requests [3][4]. Finally, ThingsBoard was used as the frontend dashboard and application layer. The whole setup and testing environment was run inside a Ubuntu Virtual Machine, where Docker Compose was used to containerize the Mosquitto and ThingsBoard infrastructure, ensuring that the environments were isolated the testing was consistent.

C. IoT Architecture Fit

Standard IoT architecture is typically categorized into multiple layers, including the Perception layer, Network or Middleware layer, and the Application layer. This platform is positioned mainly in the Middleware and Edge computing layer. It interfaces directly with the Perception layer by exposing accessible network endpoints and utilizing protocol bridges capable of receiving data from end devices. This flow of data is possible with the assumption of native IP connectivity between endpoints. Once the data is ingested, the platform acts as a gateway by first processing the payload, logging the telemetry into the database and finally forwarding the data upward to the Application layer managed by ThingsBoard.

II. PLANNING & DESIGN

A. Technology Stack and Infrastructure

The planning and design phase relied heavily on prior knowledge gathered from previous lab exercises that rely heavily on containerization. The same approach was taken for this platform, where Docker was utilized to spin up isolated environments for the external services, specifically the Eclipse Mosquitto MQTT broker and the ThingsBoard dashboard, managed by a docker-compose.yml file. This approach prevents port conflicts on the host machine and ensures consistent deployment.

The core middleware was developed in Python, utilizing the Flask web framework (app.py) to manage HTTP routing and API endpoints. For persistent local edge storage, SQLite was integrated using an initialization script (init_db.py) to create a lightweight file-based database (iot_platform.db) that does not require a dedicated backend server.

B. Decoupled Protocol Architecture

Early in the planning phase a design decision was made to decouple the protocol listeners from the main Flask application to prevent synchronous blocking issues, particularly when handling asynchronous connections. Rather than integrating all listeners into a single script as seen in Fig. 1, standalone protocol bridges were developed:

- **MQTT Adapter:** A dedicated script that uses the paho-mqtt library to subscribe to the containerized Mosquitto broker [2]. It listens for incoming sensor payloads and acts as a bridge
- **CoAP Adapter:** A separate script that runs an asynchronous aiocoap server to listen for CoAP PUT requests.

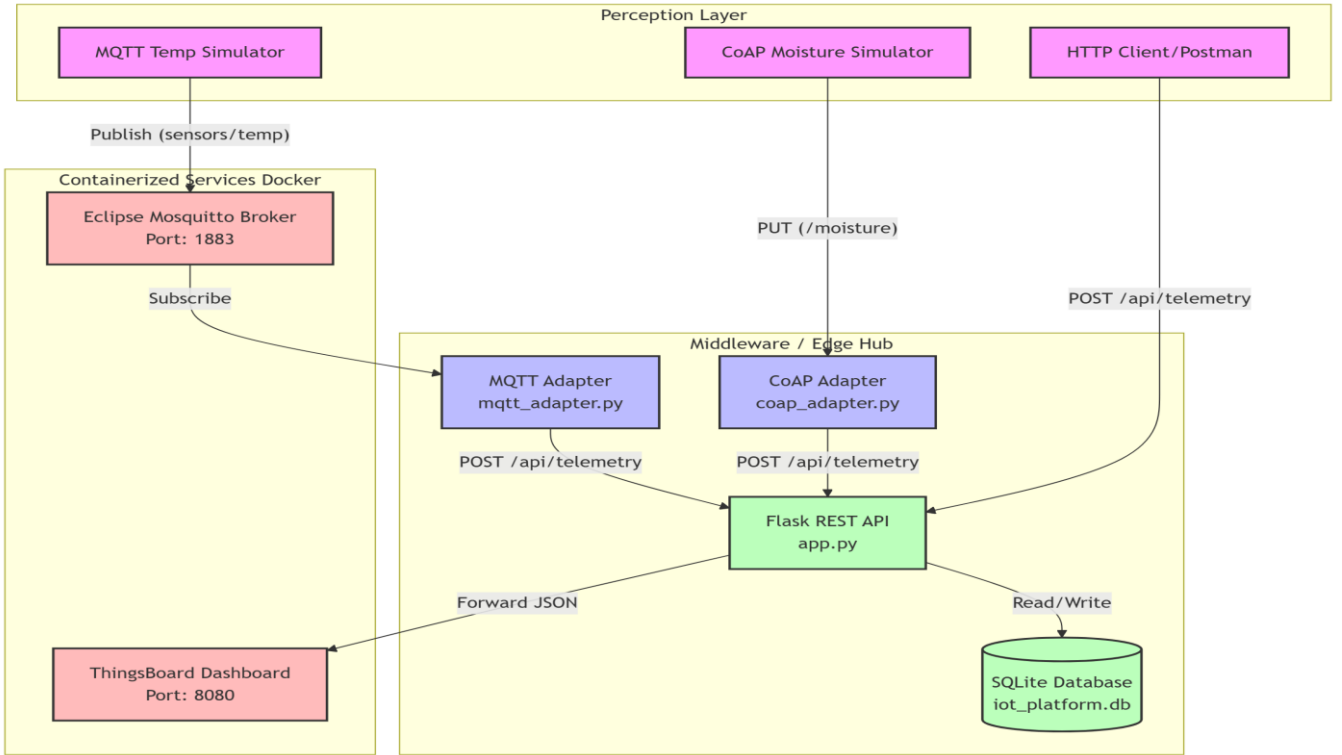


Fig. 1. Detailed system architecture block diagram illustrating the data routing of the system

Both adapters operate independently. Upon receiving the data, they translate the payload and forward it to the central Flask application via a standardized HTTP POST request. This decoupled data flow ensures that the Flask app then separately handles data processing, storage, and upstream forwarding to the Application layer.

C. Database Design

The local data processing requires retaining historical telemetry and managing device credentials securely. The SQLite database schema was designed with two primary tables: One for the devices and their metadata such as a unique device ID, device name, the protocol it uses and the ThingsBoard access token. The other table contains the telemetry itself, including sensor data and a precise timestamp.

When the middleware receives a payload, it first queries the Devices table to verify the endpoint and retrieve the associated ThingsBoard token. It then commits the payload to the Telemetry table for local persistent storage before finally formatting the data as a JSON object and forwarding it to the local ThingsBoard HTTP Telemetry API.

III. FEATURES

A. Device Registration

The platform includes a dynamic system to manage the various endpoints communicating with the middleware. Users or edge devices can register themselves via a dedicated HTTP endpoint. During registration, the device provides its identifying name, its intended communication protocol (e.g.,

MQTT, CoAP, or HTTP), and a pre-configured ThingsBoard access token. This information is securely stored in the local database, allowing the platform to dynamically route and authenticate data for each specific device without hardcoding credentials into the application logic.

B. Local Storage and Retrieval

Rather than acting solely as a pass-through proxy, the platform functions as an edge storage node. Every valid telemetry payload is intercepted and saved locally to the Telemetry table along with a precise timestamp. This feature ensures no data is lost even if the upstream connection is temporarily interrupted. Additionally, the platform provides data retrieval API endpoints, allowing developers to query the local database to find historical telemetry for any specific device.

C. Dashboard Integration

The platform features an automated forwarding mechanism that bridges the gap between the edge and the application layer. Once a payload is validated and saved locally, the middleware queries the database to retrieve the specific device's ThingsBoard token. It then strips internal routing metadata and repackages the payload into a standard JSON format, where it is automatically forwarded to the ThingsBoard HTTP Telemetry API [5]. While new devices must initially be added in the ThingsBoard UI to generate an access token, this centralized forwarding mechanism ensures that following incoming MQTT, CoAP, or HTTP data is seamlessly routed to the dashboard.

IV. API & SETUP AND TESTING

This section provides comprehensive instructions for deploying the IoT platform within an Ubuntu Virtual Machine environment and details how to interact with the core APIs for testing.

A. Environment Setup

0. Clone the project repository from: <https://github.com/mickesarro/IoT-platform-project/tree/main>
1. Navigate to the project directory and execute `docker-compose up -d`. This will pull and start the Eclipse Mosquitto broker (mapped to host port 1883) and the ThingsBoard postgres instance (web UI mapped to port 8080, internal MQTT mapped to 1884 to avoid conflicts).
2. Create and activate a Python virtual environment to isolate project dependencies:

```
python3 -m venv venv  
source venv/bin/activate
```
3. Install Dependencies: Install the required libraries via pip: *Flask*, *requests*, *paho-mqtt*, and *aiocoap*
4. Initialize Database: Run the initialization script to create the local SQLite database and its schema:

```
python3 init_db.py
```

B. Starting the Components

Because of the decoupled architecture, the core API and the protocol adapters must be started in separate terminal windows (with the virtual environment activated in each):

1. Start the central Flask REST API: `python3 app.py` (Runs on `http://127.0.0.1:5000`).
2. Start the MQTT adapter: `python3 mqtt_adapter.py` (Listens to Mosquitto on port 1883).
3. Start the CoAP adapter: `python3 coap_adapter.py` (Starts an aiocoap server on UDP port 5683).

C. API Usage & Testing

Once the system is running, users can test data flow using tools like Postman or Curl.

1. Device Registration: Before sending data, a device must be registered to obtain an internal device ID and map a ThingsBoard token.
 - Endpoint: `POST api/register`
 - Payload: `{"name": "TempSensor_1", "protocol": "MQTT", "tb token": "YOUR_SECRET_TOKEN"}` (This is the ThingsBoard token that is obtained from creating a new device in the UI <http://127.0.0.1:8080>)
 - Result: The platform returns a device ID

2. Testing **HTTP** Telemetry: Devices supporting HTTP can bypass the adapters and send data straight to the middleware
 - Endpoint: `POST api/telemetry`
 - Payload: `{"device_id": 1, "value": 25.5}`
 - Result: Data is saved locally and forwarded to ThingsBoard
3. Testing **MQTT** telemetry: To simulate a continuous stream of MQTT data, a standalone Python script `mqtt_temp_sim.py` was developed. This script connects to the Mosquitto broker and periodically publishes randomized temperature data to the `sensors/temp` topic.
 - Before running the simulator, you have to make sure that the Device ID is mapped correctly inside the `mqtt_temp_sim.py` file (ID = 1 by default)
 - Command: In the project root, running `python3 simulators/mqtt_temp_sim.py` will run the simulator and send data periodically every 5 seconds.
4. Testing **CoAP** Telemetry: Similarly, a CoAP simulator `coap_moist_sim.py` was implemented using the aiocoap library. It acts as a constrained device, continuously sending asynchronous PUT requests containing moisture data to the adapter's `coap://127.0.0.1/moisture` endpoint
 - Before running the simulator, you have to make sure that the Device ID is mapped correctly inside the `coap_moist_sim.py` file (ID = 2 by default)
5. Data Retrieval
 - To confirm the data was saved locally, users can perform a GET request to `/api/data/<device_id>`. The user can also navigate to <http://127.0.0.1:8080> and log into the ThingsBoard and check the "Latest Telemetry" tab for the corresponding device
 - Example command: `curl -X GET http://localhost:5000/api/data/3` returns with all the locally saved data on device ID = 3

V. CHALLENGES

The main challenge during the planning phase presented a significant conceptual challenge, which was designing a system that fulfilled all the project requirements such as integrating at least two distinct application-level protocols (MQTT and CoAP) without making the platform unnecessarily bloated. Ensuring that the architecture remained lightweight while supporting these communication methods required some decoupling from the core Flask application.

Another expected involved network port conflicts during the infrastructure initialization. Both the standalone Eclipse Mosquitto broker and the embedded MQTT service within the ThingsBoard container default to port 1883. When initially launching the `docker-compose.yml` file, this caused a

binding collision, preventing the containers from starting. On top of that there were initially some problems with running ThingsBoard as well. There were also problems with permissions regarding ThingsBoard internal data folder since it didn't have the needed permissions initially.

VI. FUTURE DIRECTIONS

Although the current prototype utilizes simulated payloads for testing, the platform's architecture was explicitly designed for seamless integration with physical hardware. The next phase of this project involves deploying real-life sensors that map directly to the platform's established data streams. These sensors would be for an outdoor temperature meter and an indoor soil moisture sensor for a houseplant.

Furthermore, to enhance the monitoring capabilities, there's a plan in place to use a repurposed Samsung Galaxy S3 smartphone as a dedicated edge-display terminal. This device will be mounted and configured to run continuously, showcasing the live ThingsBoard dashboard to provide immediate, always-on visibility showcasing the sensor values constantly.

VII. CONCLUSION

This project successfully demonstrated the implementation of a lightweight, multi-protocol IoT middleware platform. By utilizing a decoupled architecture, the system is able to ingest asynchronous data from both MQTT and CoAP endpoints without causing blocking issues within the central Flask application. The integration of local SQLite storage ensures the persistence of data on the edge, while the automated forwarding mechanism bridges the gap to the Application layer, which allows for immediate visualization in ThingsBoard. The containerized deployment using Docker Compose provides an effective way of maintaining an isolated environment. Ultimately, the platform meets all the requirements as well as the needed tools for testing and simulating the system. All in all the system provides a highly scalable foundation for future integration with physical sensors and display terminals.

REFERENCES

- [1] Pallets, "Using SQLite 3 with Flask," *Flask Documentation (3.0.x)*. [Online]. Available: <https://flask.palletsprojects.com/en/stable/patterns/sqlite3/>. [Accessed: May 30, 2026].
- [2] Eclipse Foundation, "Eclipse Paho MQTT Python client library," *Eclipse paho-mqtt Documentation*. [Online]. Available: <https://eclipse.dev/paho/files/paho.mqtt.python/html/index.html>. [Accessed: May 30, 2026].
- [3] Eclipse Foundation, "Eclipse Mosquitto Documentation," *Mosquitto.org*. [Online]. Available: <https://mosquitto.org/documentation/>. [Accessed: May 30, 2026].
- [4] C. Amstüss, "aiocoap - The Python CoAP library," *Read the Docs*. [Online]. Available: <https://aiocoap.readthedocs.io/en/latest/>. [Accessed: May 30, 2026].
- [5] ThingsBoard Authors, "Working with telemetry data," *ThingsBoard Documentation*. [Online]. Available: <https://thingsboard.io/docs/user-guide/telemetry/>. [Accessed: May 30, 2026].